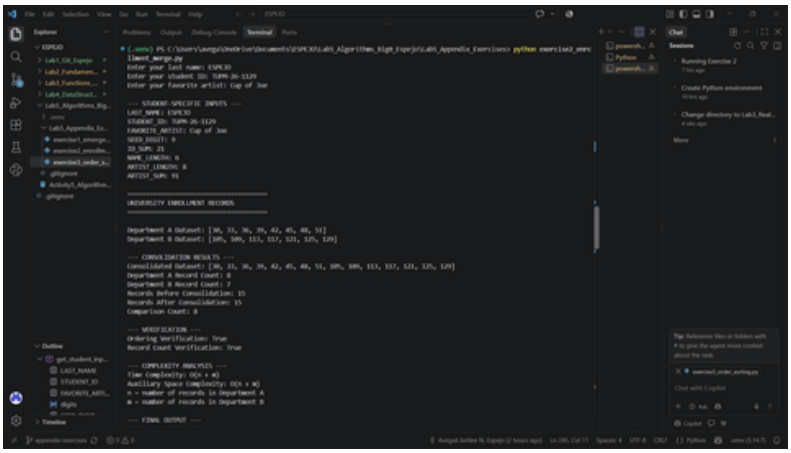
ESPEJO, AVEGAIL ASHLEE N.
BSECE - 1C

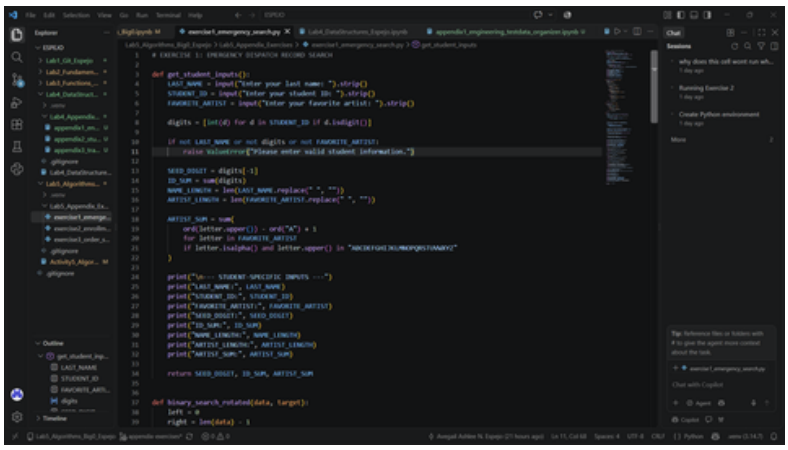
Activity 5
ALGORITHMS & BIG O

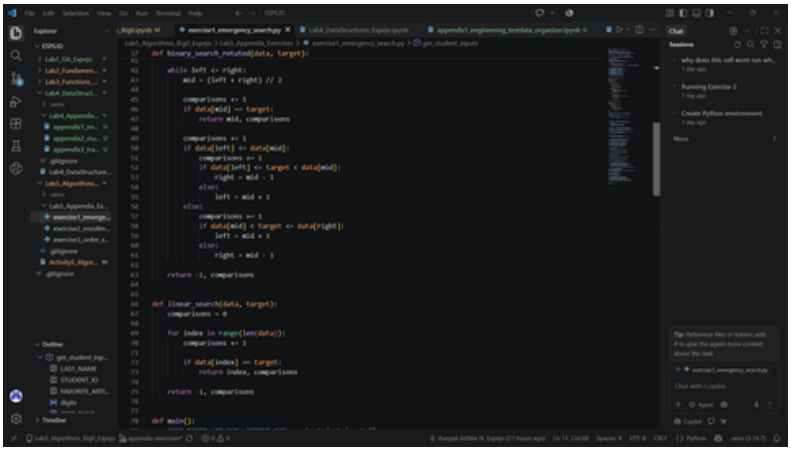
APPENDIX 1: Emergency Dispatch Record Search



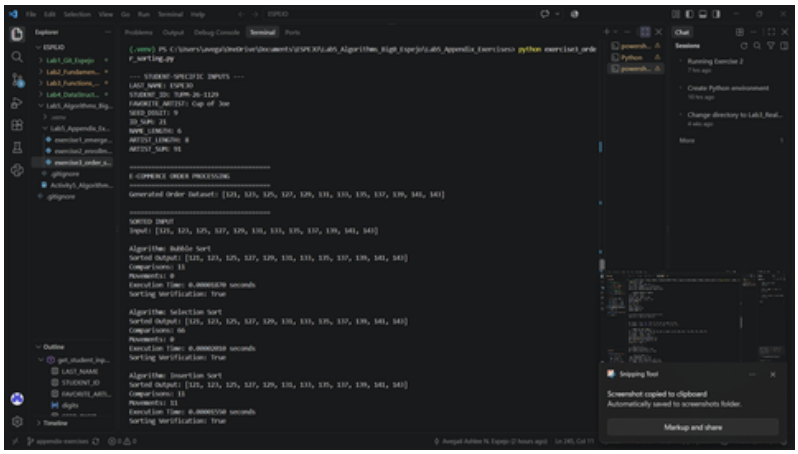


```
--- FINAL OUTPUT ---
Original Dataset: [121, 128, 135, 142, 149, 156, 163, 170, 177, 184, 191, 198]
Rotated Dataset: [184, 191, 198, 121, 128, 135, 142, 149, 156, 163, 170, 177]
Target Incident: 149
Binary Search Comparisons: 10
Linear Search Comparisons: 8
```

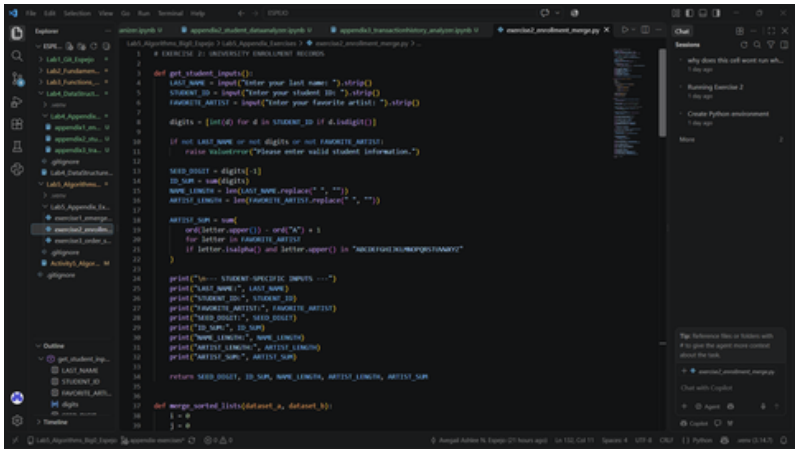


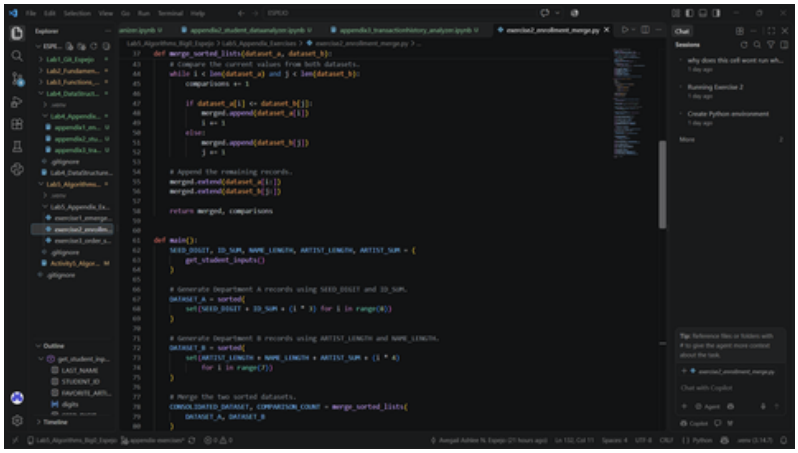


APPENDIX 2: University Enrollment Records



```
--- FINAL OUTPUT ---
Department A: [30, 33, 36, 39, 42, 45, 48, 51]
Department B: [105, 109, 113, 117, 121, 125, 129]
Consolidated Dataset: [30, 33, 36, 39, 42, 45, 48, 51, 105, 109, 113, 117, 121, 125, 129]
Comparison Count: 8
Correctly Ordered: True
All Records Retained: True
```

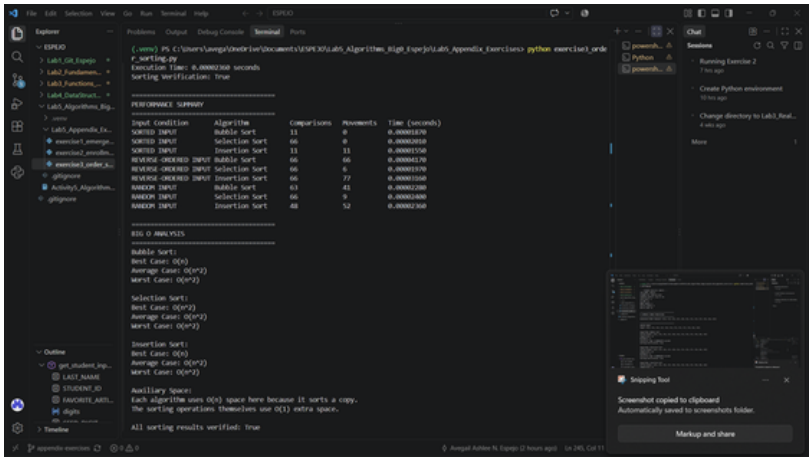




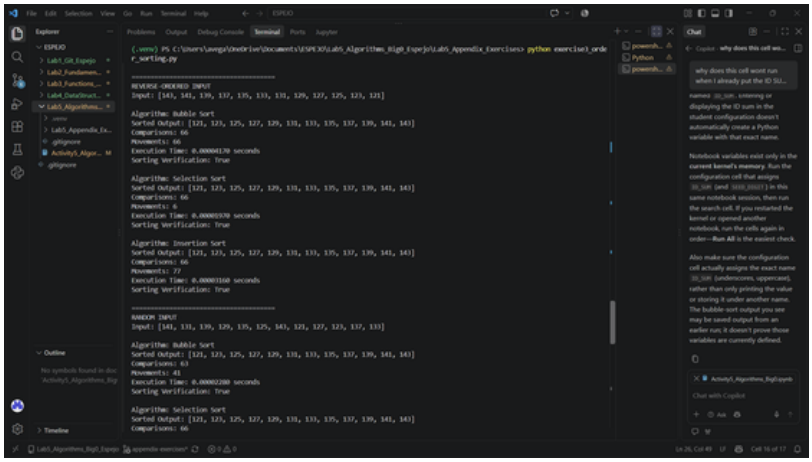


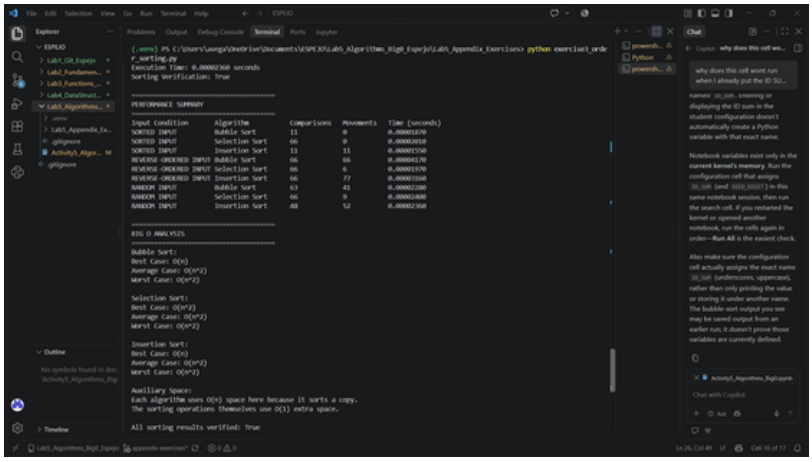


APPENDIX 3: E-Commerce Order Processing

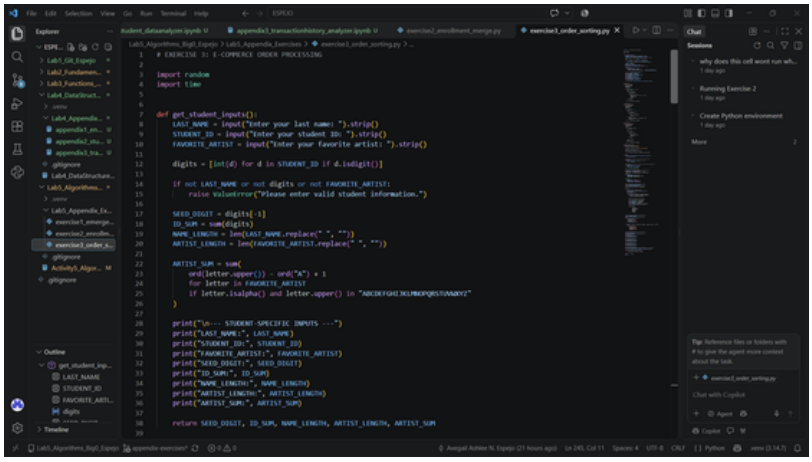


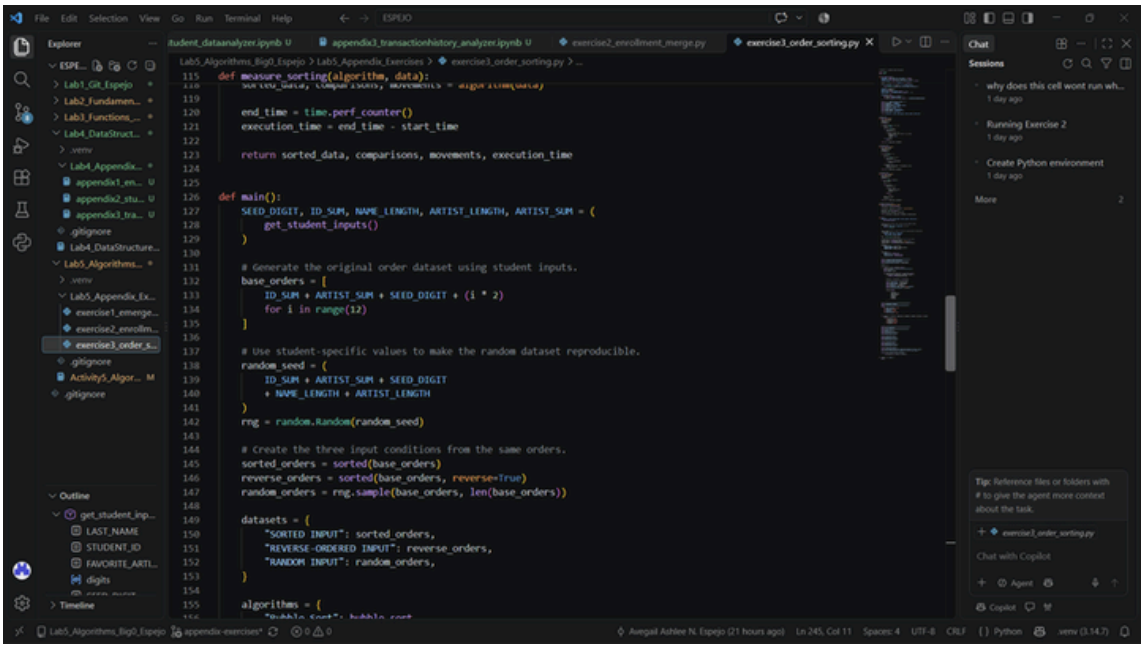












```python
def measure_sorting(algorithm, data):
    end_time = time.perf_counter()
    execution_time = end_time - start_time

    return sorted_data, comparisons, movements, execution_time


def main():
    SEED_DIGIT, ID_SUM, NAME_LENGTH, ARTIST_LENGTH, ARTIST_SUM = (
        get_student_inputs()
    )

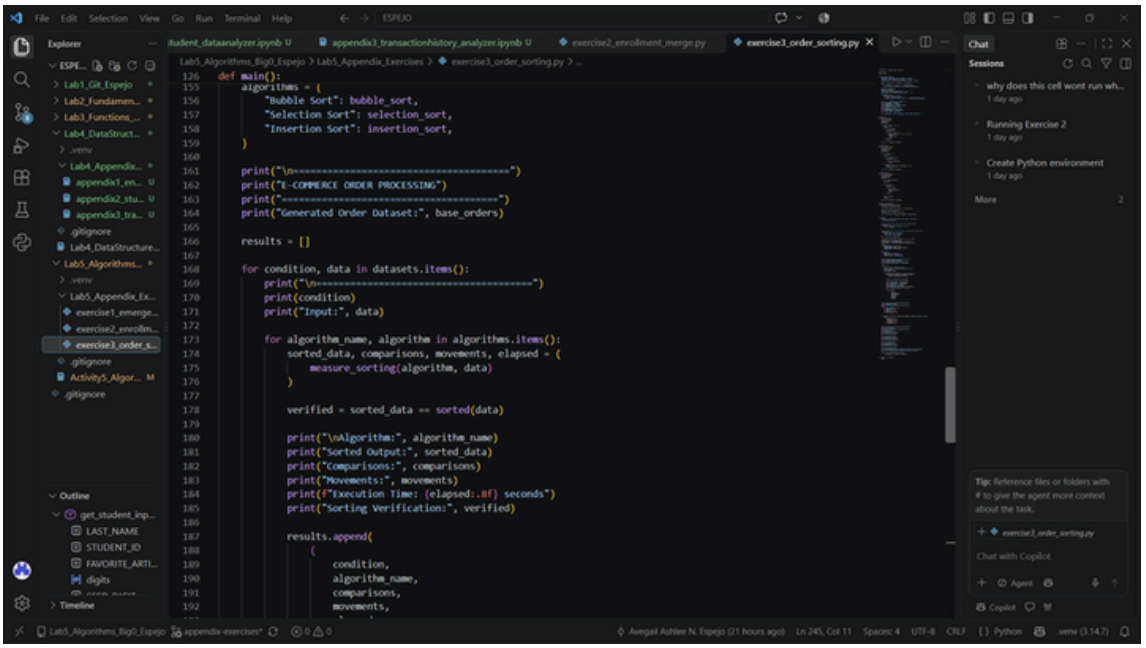
    # Generate the original order dataset using student inputs.
    base_orders = [
        ID_SUM + ARTIST_SUM + SEED_DIGIT + (i * 2)
        for i in range(12)
    ]

    # Use student-specific values to make the random dataset reproducible.
    random_seed = (
        ID_SUM + ARTIST_SUM + SEED_DIGIT
        + NAME_LENGTH + ARTIST_LENGTH
    )
    rng = random.Random(random_seed)

    # Create the three input conditions from the same orders.
    sorted_orders = sorted(base_orders)
    reverse_orders = sorted(base_orders, reverse=True)
    random_orders = rng.sample(base_orders, len(base_orders))

    datasets = {
        "SORTED INPUT": sorted_orders,
        "REVERSE-ORDERED INPUT": reverse_orders,
        "RANDOM INPUT": random_orders,
    }

    algorithms = {
```



```python
def main():
    algorithms = {
        "Bubble Sort": bubble_sort,
        "Selection Sort": selection_sort,
        "Insertion Sort": insertion_sort,
    }

    print("\n" + "=" * 70)
    print("E-COMMERCE ORDER PROCESSING")
    print("=" * 70)
    print("Generated Order Dataset:", base_orders)

    results = []

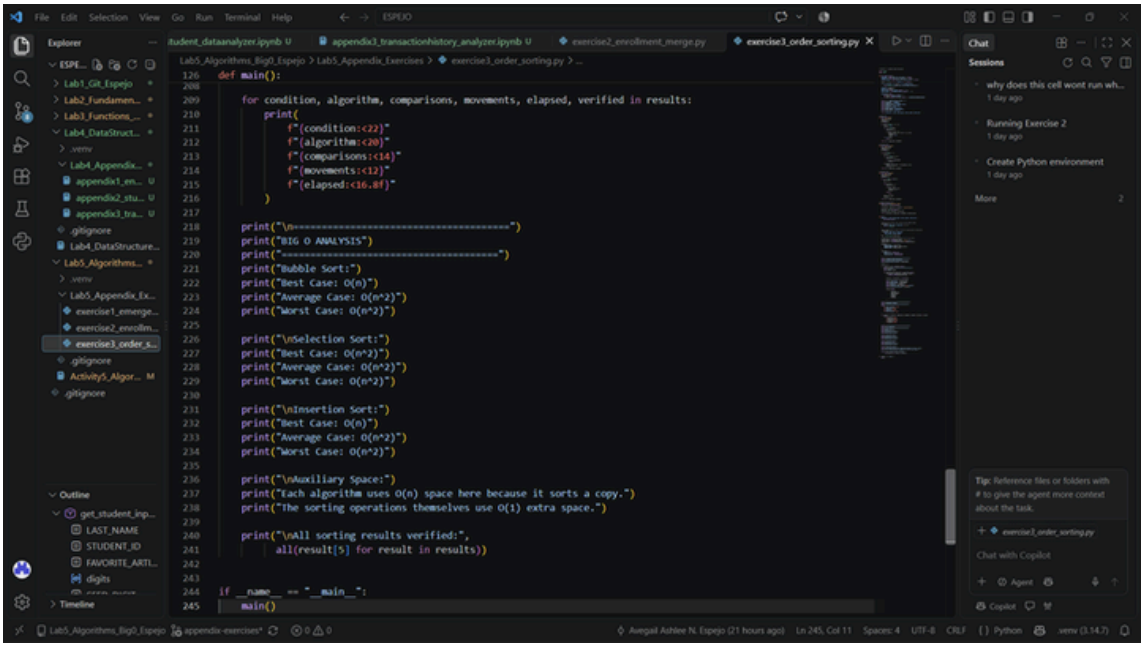
    for condition, data in datasets.items():
        print("\n" + "=" * 70)
        print(condition)
        print("Input:", data)

        for algorithm_name, algorithm in algorithms.items():
            sorted_data, comparisons, movements, elapsed = (
                measure_sorting(algorithm, data)
            )

            verified = sorted_data == sorted(data)

            print("\nAlgorithm:", algorithm_name)
            print("Sorted Output:", sorted_data)
            print("Comparisons:", comparisons)
            print("Movements:", movements)
            print(f"Execution Time: {elapsed:.8f} seconds")
            print("Sorting Verification:", verified)

            results.append(
                (
                    condition,
                    algorithm_name,
                    comparisons,
                    movements,
```



```python
def main():

    for condition, algorithm, comparisons, movements, elapsed, verified in results:
        print(
            f"{condition:<22}"
            f"{algorithm:<20}"
            f"{comparisons:<14}"
            f"{movements:<12}"
            f"{elapsed:<16.8f}"
        )

    print("\n" + "=" * 70)
    print("BIG O ANALYSIS")
    print("=" * 70)
    print("Bubble Sort:")
    print("Best Case: O(n)")
    print("Average Case: O(n^2)")
    print("Worst Case: O(n^2)")

    print("\nSelection Sort:")
    print("Best Case: O(n^2)")
    print("Average Case: O(n^2)")
    print("Worst Case: O(n^2)")

    print("\nInsertion Sort:")
    print("Best Case: O(n)")
    print("Average Case: O(n^2)")
    print("Worst Case: O(n^2)")

    print("\nAuxiliary Space:")
    print("Each algorithm uses O(n) space here because it sorts a copy.")
    print("The sorting operations themselves use O(1) extra space.")

    print("\nAll sorting results verified:",
          all(result[5] for result in results))


if __name__ == "__main__":
    main()
```